

## PeltOS-Spinlock的ARM版本



PeltOS札记。中文名兽皮的这款裸机操作系统在网上直播创作过程。之所以取名兽皮是因为总比裸机好点，等那一天能耐见长，再整点布衣穿穿。老百姓过日子，柴米油盐，习惯了，也就没有宏大叙事了。

Spinlock叫做自旋锁，差不多是所有操作系统躲不过的事情。

按照惯例，自旋锁的行为这样描述：锁有两种状态，一种是空闲free，一种是锁住locked。程序行为是检查一把锁，如果是free，锁上变成locked，然后做一件独占的事情，做完后开锁，变成free。如果发现锁已经锁上了，那就在这里死等，直到锁开了，把锁锁上，做一件独占的事情，做完后开锁。

这样从检查锁直到做完事情后开锁都必须满足原子性操作前提。啥叫原子性呢？当初建立这样概念时，大家觉得原子是不可分最小单元，就起了这样的名字Atomic，意思是这些操作①所涉及的内存要独占，②动作要一气呵成。

啥叫内存独占呢？比方记录锁状态的这个标志就属于必须由持锁的一方才能修改，一般情况下，大家会约定还有些地方也只能由持锁方修改。这就叫内存的独占性。

什么叫一气呵成呢？这些操作不能被打断。至于啥叫不能打断后面会针对角色详细谈。

那些角色呢？主程序、中断、DMA还有多核。

多核这个问题反而是自旋锁里面做简单的问题，因为，多核之间不存在谁打断谁的问题，只要遵守约定，拿到锁的控制权，就没事了。拿不到，别的核一会儿肯定会开锁，死等是没问题的。

DMA会在程序运行过程中对内存进行操作。由于约定独占的内存不应该出

现在DMA可以访问到的位置，所以这一招就排除了DMA。

剩下的就是中断了。如果spinlock一定不会在中断中访问，那么就不会出现被打断的现象，那么对于单核的系统，这把锁意义也就不大了。其实对于传统计算机而言，线程和进程之间的切换等效于中断中访问这把锁。因此，spinlock的操作一定要把中断关上，才能保证一气呵成。进入中断以后，系统关闭中断了，要么中断程序显式的打开中断，要么中断程序退出，才能由新的中断发生。

这次讲的是ARM体系，采用的是新唐Nuvoton的M463处理器，CortexM4架构。

```
/* Raw atomic try-acquire: return 1 if success, 0 if busy */
static inline int32_t __piSpinlockRaw(volatile int32_t* addr) {
    uint32_t old_val;
    uint32_t new_val;
    uint32_t status;
    do {
        __asm volatile(
            "ldrex %0, [%3]      \n\t" /* old_val = *addr */
            "cmp    %0, #0      \n\t" /* check if unlocked */
            "bne    1f          \n\t" /* if locked, jump to fail */
            "mov     %1, #1      \n\t" /* new_val = 1 */
            "strex %2, %1, [%3]  \n\t" /* try store */
            "b       2f          \n\t"
            "1:              \n\t"
            "mov     %2, #2      \n\t" /* mark as busy */
            "2:              \n\t"
            : "=&r"(old_val), "=&r"(new_val), "=&r"(status)
            : "r"(addr)
            : "cc", "memory"
        );
        if (status == 2) {
            return 0; /* already locked */
        }
    } while (status != 0);
    return 1;
}
```

在x86架构上，有一条指令是天然的spinlock候选指令：

```
xchg ax, word ptr[lock]。
```

这条指令在一条指令内完成内存的读修改写回操作，单核原子性没问题。多核之间，程序在指令前加一个前缀指令ESC，然后，会在总线访问时通过总线仲裁的方式获得独占性，这些都是40年前渔夫学到的。后来x86也有了缓存，情况就复杂多了，但路子也就这样，渔夫等到写一个x86的兽皮会去研究这个问题。

ARM秉承现代计算机精简指令集的策略，没有在一条指令内完成两次内存操作的指令，没有指令层面的原子性，这也难不倒ARM的同志们。M460系列就支持Exclusive Monitor功能，也就是内存独占监视器。当执行LDREX指令时，就是通知Exclusive Monitor给看着这个地址，是不是我读了这个内存以后，还有别人写入这个地址。监视到啥时呢？STREX指令时，有个寄存器会接收来自Exclusive Monitor的汇报，就是在从执行LDREX指令开始到STREX指令完成这段时间内由没有别人写入过这个地址。谁会写入呢，别的核，DMA，如果中断时打开的，中断写入也有可能。

首先前面的这个循环可不是死等的循环，而是保持原子操作的循环。第一条指令：

```
LDREX %0, [%3]
```

这句话启动Exclusive Monitor，并且把addr(%3)作为指针的内存取到old\_val(%0)中，这个是编译器自动安排的寄存器，具体哪一个不需要关心。

```
cmp    %0, #0
bne    1f
```

这句话是说如果锁被占用则跳转到1。如果old\_val不等于0跳转到下面这条。

```
"mov    %2, #2        \n\t" /* mark as busy */
```

```
status=2
```

```
"mov    %1, #1        \n\t" /* new_val = 1 */
"strex %2, %1, [%3]    \n\t" /* try store */
"b      2f            \n\t"
```

这些话加起来就是把1存入addr(%3)作为指针的内存中，同时Exclusive Monitor把监视结果写入status(%2)中。

```
if (status == 2) {
    return 0; /* already locked */
}
```

到这里判断时，status可能有三种情况：0表示监视到有人打断这个原子操作，没有写入成功；1写入成功，拿到锁；2锁原来就是锁上的。这个尾缀\_RAW的函数遇到锁被占用的情况，立即退出结果为0。

```
} while (status != 0);
```

当status==1时，没原子成功，继续尝试。

```
return 1;
```

这句话是说，拿到锁了，返回1。

为啥会有这个不死等的锁呢？这是因为，对于兽皮这样没线程的全异步的系统，这个不死等的锁用处也不少。同时这个raw的半成品后面会组织成真正的spinlock。

```
static inline void pelt_spinlock_lock(volatile pelt_spinlock_t* s){
    uint32_t primask;
    for (;;) {
        primask = __piDisableIRQSave(); // Disable interrupts and save state
        if (__piSpinlockRaw(&s->lock)) {
            s->irq_mask = (int32_t)primask;
            __piDMB();
            return;
        }
        /* Restore IRQ state before retrying to avoid deadlock on single-core */
        __piSetPRIMASK(primask);
    }
}
```

这个函数提供spinlock自旋锁的死等功能。首先关闭中断，关中断之前保存状态到primask中，试图上锁（调用\_\_piSpinlockRaw）如果拿到锁，把中断状态保存起来，退出。如果没拿到锁，恢复中断状态，回头再试，死等。

这里说一下恢复中断状态的事。进入这个函数之前，中断可以是开着的，也可以是关着的。所以每次保存当前的中断状态再关中断，而不是关中断、开中断这种莽撞的操作。

```
static inline uint32_t __piDisableIRQSave(void) {
    uint32_t primask;
    __asm__ volatile ("mrs %0, primask" : "=r" (primask));
    __asm__ volatile ("cpsid i" : : "memory");
    return primask;
}
```

我们现在使用的Cortex很多版本中，MSR指令读取中断屏蔽状态，cpsid则是关中断。有意思的是memory这个词是提醒编译器，我动了内存，你别乱优化，按我的原样就行。

渔夫忍不住插一句。经常有自媒体说C++不安全，很快会被啥啥语言取代。当你懂一些计算机时，比方这个自旋锁，你就会发现，你连编译器的行为都需要干预的编程模式，语言算啥？编程不是按照编程语言的语法写出来那么简单的。很多的幻觉来自于知道的太少。

比较一下死等与拿到锁退出的差异，死等之前要恢复中断状态再去查询锁，而拿到锁以后，把中断屏蔽状态存起来就退出。这也就意味着加锁以后直到释放锁这段时间中间这段时间中断始终是关闭的。

```
static inline void pelt_spinlock_unlock(volatile pelt_spinlock_t* s) {
    __piDMB();
    s->lock = 0;
    __piDMB();
    __piSetPRIMASK((uint32_t)s->irq_mask);
}
```

倒叙一下，自旋锁的定义

```
typedef struct {
    volatile int32_t lock; // 0: free, != 0: locked, need to initialize lock to 0
    volatile int32_t irq_mask; //
} pelt_spinlock_t;
```

初始化：

```
void pelt_spinlock_init(pelt_spinlock_t* lock) {
    lock->lock = 0; // Initialize the lock to unlocked state
    lock->irq_mask = 0; // Initialize the IRQ mask to 0
    __piDMB(); // Ensure memory operations are completed before proceeding
}
```

渔夫一直在思考为啥Linux不用C++，一直用C？没想明白，所以在自旋锁这一点，用类定义也很顺畅，但是这一版看官见到的仍然是标准C的自旋锁。啥时候会改成类，等渔夫测试一下两者的异同再说。

渔夫请Gemini给设计了题图，兽皮的口号是，A native fully asynchronous and objects-oriented。注意and是native fully asynchronous与objects-oriented并列，也就是说不是fully objects-oriented。原生全异步，这个每个环节都秉承，面向对象这个有例外，这个自旋锁目前就是例外。

兽皮的某些内容可以在<https://github.com/Petereetic>上看到，因为不懂开

源所以没有全部公开。

愿意与渔夫切磋的看官，微信加PeterInnoStudio